

Three.js Developer Assessment

Take-Home Assignment

Company: SurgePV Design Studio **Position:** Three.js Developer **Time Estimate:** 2-3 hours **Submission:** GitHub repository or ZIP file with code + screenshots

Overview

SurgePV Design Studio is a professional solar panel design application built with React and Three.js. The 3D engine runs in a Web Worker for performance. We use an Entity-Component-System (ECS) architecture where **Systems** handle specific functionality like selection, highlighting, and drawing.

Your task is to implement a **Measurement Tool** that allows users to measure distances in 3D space, following engineering drawing standards.

Architecture Context

System Pattern

All interactive features are implemented as Systems. Here's the base class:

```

// System.ts - Base class for all systems
export default class System {
  private static registry: WeakMap<Function, System> = new WeakMap();
  protected enabled: boolean = true;
  protected modeChangeCallback?: (mode: string) => void;

  protected constructor() {
    this.enabled = true;
  }

  /**
   * Singleton factory - creates instance if not exists, returns existing otherwise
   */
  static createReference(this: any, ...args: any[]): any {
    const existing = System.registry.get(this);
    if (existing) return existing as any;
    const instance = new (this as any)(...args);
    System.registry.set(this, instance);
    return instance;
  }

  static getReference(this: any): any {
    const existing = System.registry.get(this);
    if (!existing) {
      throw new Error(`${this.name} instance not created. Call createReference first.`);
    }
    return existing as any;
  }

  /** Called when system is initialized */
  init(_dependencies: any): void {}

  /** Called every frame */
  update(_deltaTime: number): void {}

  /** Called when system's mode becomes active */
  activate(): void {}

  /** Called when system's mode becomes inactive */
  deactivate(): void {}

  /** Full cleanup when system is destroyed */
  dispose(): void {}

  /**
   * Handle events from the event system
   * @returns true if event was handled (stops propagation), false otherwise
   */
  handleEvent(_category: string, _eventData: any): boolean {
    return false;
  }
}

```

Event Types

Events are organized by category:

```
export const EVENT_TYPES = {
  MOUSE: {
    CATEGORY: 'MOUSE',
    CLICK: 'MOUSE_CLICK',
    MOVE: 'MOUSE_MOVE',
    DOWN: 'MOUSE_DOWN',
    UP: 'MOUSE_UP',
  },
  KEYBOARD: {
    CATEGORY: 'KEY',
    KEY_DOWN: 'KEY_DOWN',
    KEY_UP: 'KEY_UP',
  },
  UI: {
    CATEGORY: 'UI',
    MEASUREMENT: {
      START: 'UI_MEASUREMENT_START',
      COMPLETE: 'UI_MEASUREMENT_COMPLETE',
      CANCEL: 'UI_MEASUREMENT_CANCEL',
      HISTORY_UPDATE: 'UI_MEASUREMENT_HISTORY_UPDATE',
    },
  },
};
```

Event Data Structures

```
// Mouse click event data
interface MouseClickEventData {
  type: 'MOUSE_CLICK';
  position: { x: number; y: number }; // Screen coordinates
  worldPosition: THREE.Vector3; // 3D world position (raycasted)
  button: 0 | 1 | 2; // Left, Middle, Right
  shiftKey: boolean;
  ctrlKey: boolean;
}

// Keyboard event data
interface KeyboardEventData {
  type: 'KEY_DOWN' | 'KEY_UP';
  key: string; // e.g., 'Escape', 'Enter', 'Shift'
  code: string; // e.g., 'Escape', 'Enter', 'ShiftLeft'
}
```

Task: Implement MeasurementSystem

Requirements

Create a MeasurementSystem that:

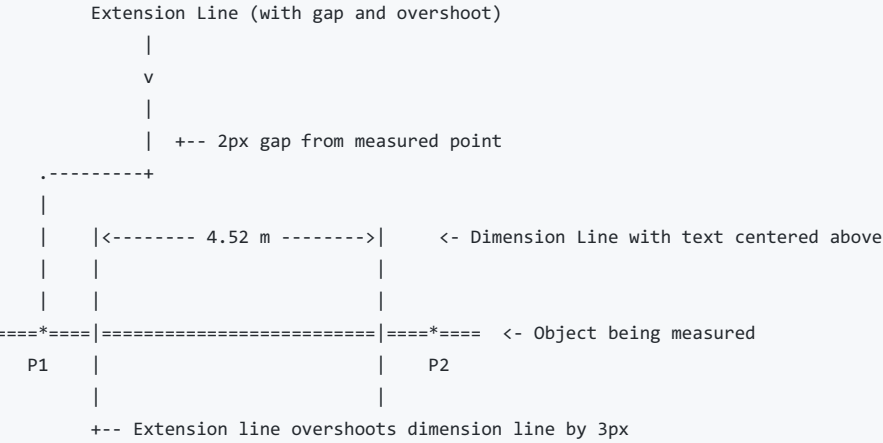
- 1. **Activates** when the measurement tool is selected
- 2. **First click** sets the start point
- 3. **Mouse move** shows a preview line to cursor position
- 4. **Second click** completes the measurement
- 5. **Escape key** cancels the current measurement
- 6. **Deactivate** cleans up all visual elements

Visual Standards (CRITICAL)

Your measurement visualization **must** follow engineering drawing standards:

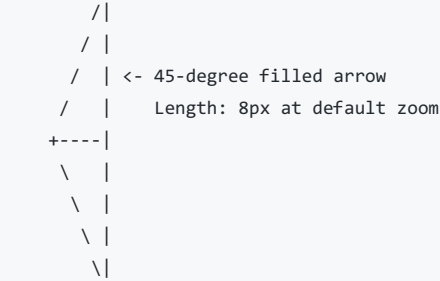
STANDARD ENGINEERING DIMENSION LINE

=====



ARROW TERMINATORS (Detail)

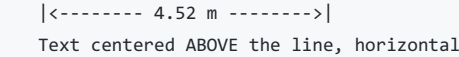
=====



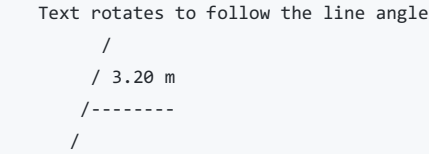
TEXT PLACEMENT

=====

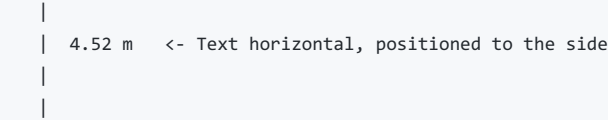
Horizontal line (0-30 degrees from horizontal):



Angled line (30-60 degrees):



Steep line (60-90 degrees):



Specifications Table

Element	Specification
Dimension Line	Solid line, color: #2563eb
Extension Lines	Same color, 2px gap from point, 3px overshoot past dimension line
Arrow Terminators	Filled triangles, 45-degree, 8px length at default zoom

Text Element	Monospace font, color: #1e40af Specification
Text Background	White (rgba(255,255,255,0.9)), 4px padding, 2px border-radius
Text Position	Centered above line, 6px gap from dimension line
Precision	2 decimal places with unit: "4.52 m"
Preview Line	Dashed line while measuring, color: #94a3b8

Starter Code

```
// MeasurementSystem.ts
import System from './baseClasses/System';
import * as THREE from 'three';

export const MEASUREMENT_EVENTS = {
  CATEGORY: 'MEASUREMENT',
  START: 'MEASUREMENT_START',
  POINT_ADDED: 'MEASUREMENT_POINT_ADDED',
  COMPLETE: 'MEASUREMENT_COMPLETE',
  CANCEL: 'MEASUREMENT_CANCEL',
} as const;

interface Measurement {
  id: string;
  startPoint: THREE.Vector3;
  endPoint: THREE.Vector3;
  distance: number;
  lineGroup: THREE.Group; // Contains all visual elements
}

export class MeasurementSystem extends System {
  private scene: THREE.Scene | null = null;
  private camera: THREE.Camera | null = null;

  // State
  private isActive: boolean = false;
  private isMeasuring: boolean = false;
  private startPoint: THREE.Vector3 | null = null;

  // Visual elements
  private previewLine: THREE.Line | null = null;
  private measurements: Map<string, Measurement> = new Map();

  // Reusable materials (for memory efficiency)
  private dimensionLineMaterial: THREE.LineBasicMaterial | null = null;
  private previewLineMaterial: THREE.LineDashedMaterial | null = null;

  constructor() {
    super();
  }

  init(dependencies: { scene: THREE.Scene; camera: THREE.Camera }): void {
    this.scene = dependencies.scene;
    this.camera = dependencies.camera;
    this.initMaterials();
  }

  private initMaterials(): void {
    // TODO: Initialize reusable materials
  }
}
```

```

handleEvent(category: string, eventData: any): boolean {
    if (!this.isActive) return false;

    // TODO: Handle MOUSE.CLICK for adding points
    // TODO: Handle MOUSE.MOVE for preview line
    // TODO: Handle KEYBOARD.KEY_DOWN for Escape to cancel

    return false;
}

activate(): void {
    // TODO: Enable measurement mode
    // - Set isActive = true
    // - Show cursor indicator
    // - Reset any partial measurement state
}

deactivate(): void {
    // TODO: Clean up when mode changes
    // - Cancel any in-progress measurement
    // - Remove preview line
    // - Keep completed measurements visible? (design decision)
}

dispose(): void {
    // TODO: Full cleanup
    // - Remove ALL measurements from scene
    // - Dispose ALL geometries and materials
    // - Clear maps
}

// ===== HELPER METHODS TO IMPLEMENT =====

private generateId(): string {
    return `measurement-${Date.now()}-${Math.random().toString(36).substr(2, 9)}`;
}

private handleClick(worldPosition: THREE.Vector3, shiftKey: boolean): void {
    // TODO: Implement click handling
    // First click: set start point, create preview line
    // Second click: complete measurement, create dimension line group
}

private handleMouseMove(worldPosition: THREE.Vector3): void {
    // TODO: Update preview line to follow cursor
}

private cancelMeasurement(): void {
    // TODO: Cancel in-progress measurement
    // - Remove preview line
    // - Reset state
}

private completeMeasurement(endPoint: THREE.Vector3): void {
    // TODO: Create the full dimension line visualization
    // - Create line group with all elements
    // - Add to scene and measurements map
}

private createDimensionLineGroup(
    start: THREE.Vector3,
    end: THREE.Vector3
): THREE.Group {
    // TODO: Create a Group containing:
    // 1. Main dimension line (between arrows)

```

```

// 2. Extension lines (from points to dimension line)
// 3. Arrow terminators (at both ends)
// 4. Text sprite with distance label

const group = new THREE.Group();

// Your implementation here...

return group;
}

private createArrowHead(
  position: THREE.Vector3,
  direction: THREE.Vector3
): THREE.Mesh {
  // TODO: Create a filled arrow triangle
  // - 45-degree angle
  // - 8px length at default zoom
  // - Pointing in the given direction
}

private createTextSprite(
  text: string,
  position: THREE.Vector3
): THREE.Sprite {
  // TODO: Create text label using Canvas + Sprite
  // - Monospace font
  // - White background with padding
  // - Proper positioning above the line
}

private calculateDistance(p1: THREE.Vector3, p2: THREE.Vector3): number {
  return p1.distanceTo(p2);
}

private formatDistance(distance: number): string {
  return `${distance.toFixed(2)} m`;
}

private getLineAngle(start: THREE.Vector3, end: THREE.Vector3): number {
  // TODO: Calculate angle of line from horizontal (in degrees)
  // Used for text rotation decisions
}

// ===== CLEANUP METHODS =====

private removeMeasurement(id: string): void {
  // TODO: Remove a specific measurement
  // - Remove group from scene
  // - Dispose all geometries in the group
  // - Remove from map
}

private clearAllMeasurements(): void {
  // TODO: Remove all measurements
}

private disposeGroup(group: THREE.Group): void {
  // TODO: Recursively dispose all geometries and materials in a group
}
}

```

Required

- 1. **Complete MeasurementSystem.ts** - Full implementation following the pattern above
- 2. **Screenshots** showing:
 - A horizontal measurement (demonstrating proper text placement)
 - An angled measurement (30-60 degrees, showing text rotation)
 - A steep/vertical measurement (showing text flip behavior)
 - Multiple measurements on screen simultaneously
- 3. **Brief write-up** (max 1 page) explaining:
 - Your approach to text rotation based on line angle
 - How you handle memory management for geometries/materials
 - Any design decisions you made that weren't specified

Bonus (Optional)

- 1. **Snap-to-increment:** Hold Shift to snap to 0.5m increments, Ctrl for 0.1m
- 2. **Chain dimensioning:** Double-click to continue adding points:

```
|<-- 2.00 m -->|<-- 3.50 m -->|<-- 1.25 m -->|
*-----*-----*-----*
```

- 3. **Measurement deletion:** Click on a measurement to select it, Delete key to remove

Evaluation Criteria

Criteria	Weight	Description
Visual Accuracy	35%	Dimension lines match engineering standards. Arrows, extension lines, and text placement are correct.
Code Quality	25%	Follows the System pattern. Clean, readable code. Proper TypeScript types.
Memory Management	25%	No leaks. Proper disposal of all Three.js objects. Reusable materials.
Edge Cases	15%	Text rotation works correctly. Handles rapid clicks. Escape cancels properly.

Questions?

If any requirements are unclear, document your assumptions in the write-up. We value candidates who identify ambiguity and make reasonable decisions.

Submission

Send your submission to: Make a google drive folder, email it to hr@heavendesigns.in and send the folder link with Sharing permissions activated on internshala.

Include:

- Source code (.ts files)
- Screenshots (PNG or JPG)
- Write-up (PDF or Markdown)

Deadline: [3 days from receipt]

Good luck!